Sistema de pedidos por QR Code — Restaurante Bom Paladar

Oct 19, 2026 · @Insight Creative Studio

Resumo

Cada mesa do Bom Paladar tem um QR Code próprio; o cliente lê, vê o cardápio no telemóvel, escolhe e envia o pedido, que aparece em tempo real no painel do administrador, já identificado pela mesa. Não precisa de app, conta nem login para o cliente.

Decisões que tomei por ti (diz se quiseres mudar alguma):

- **Sem pagamento online.** O cliente pede e paga no fim, no caixa ou ao empregado. O sistema nunca processa pagamentos, nem agora nem mais tarde.
- **QR Code com código secreto, não com o número da mesa.** O link é tipo `/m/k7x9q2`, não `/mesa/3`. Assim ninguém consegue fazer pedidos para outra mesa só mudando o número no endereço.
- **Moeda: Meticais (MT).** Fácil de mudar.
- **Começa com 10 mesas**, e o administrador adiciona ou remove mesas quando quiser.
- **Uma conta de administrador** com email e senha. Mais contas (cozinha, empregado) ficam para depois.
- **Notificação em tempo real:** pedido novo toca um som e aparece no topo do painel, sem precisar de atualizar a página.

Stack recomendada

Vite com React e Supabase, publicado na Vercel, é a escolha certa aqui: o site é só frontend e o Supabase faz de backend (base de dados, login e tempo real), sem servidor próprio para manter.

Peça	Ferramenta	Porquê
Site (cliente + admin)	Vite + React + TypeScript + React Router	Uma só base de código para as duas partes
Visual	Tailwind CSS	Rápido e fica bom no telemóvel
Base de dados	Supabase (Postgres)	Tabelas simples e relacionais

Peça	Ferramenta	Porquê
Tempo real	Supabase Realtime	O pedido aparece no painel sem atualizar
Login do admin	Supabase Auth	Email e senha, sem inventar segurança
Imagens dos pratos	Supabase Storage	O admin carrega a foto pelo painel
QR Codes	Biblioteca `qrcode`	Gera os QR no próprio sistema, prontos a imprimir
Hospedagem	Vercel	Site estático com link público e HTTPS automático

Se preferires construir na Lovable, serve o mesmo desenho, mas para este caso o Claude Code dá mais controlo sobre o tempo real e as regras por mesa.

Funcionalidades

São duas partes, como pediste: a parte do cliente (pública) e o painel do administrador (com senha).

Parte do cliente

- Lê o QR Code da mesa e abre o cardápio, já a dizer "Mesa 3" no topo.
- Cardápio por categorias (Pratos, Bebidas, Sobremesas…), com foto, descrição e preço.
- Toca num prato, escolhe a quantidade e escreve uma nota ("sem cebola", "sem gelo", "mais uma água").
- Carrinho com o resumo e o total; botão **Enviar pedido**.
- Ecrã de confirmação, e a possibilidade de fazer novos pedidos na mesma mesa.
- Vê os pedidos que já fez nessa mesa e o estado de cada um (Recebido, Em preparação, Entregue).
- Pratos esgotados aparecem cinzentos e não dá para pedir.

Painel do administrador

- Login com email e senha.
- **Visão geral das mesas:** cartões Mesa 1, Mesa 2… Mesa 10, cada um a mostrar se está livre ou com pedidos, o total em aberto e a hora do último pedido.

- **Detalhe da mesa:** todos os pedidos, com hora, itens, quantidade, notas e estado; botões para mudar o estado (Recebido → Em preparação → Entregue).
- **Histórico:** lista de todos os pedidos por data, filtrável por mesa; botão para fechar a mesa depois de paga, que a deixa livre para o próximo cliente.
- **Adicionar, renomear ou remover mesas**, com o QR Code de cada uma pronto a imprimir (uma página com todos os QR, uma mesa por cartão).
- **Gerir cardápio:** categorias, pratos, preço, foto, e marcar como esgotado.
- Alerta sonoro e destaque visual quando chega um pedido novo.

Fora desta versão, para não complicar: contas para empregados e cozinha, impressão automática de talão. Pagamento online fica de fora de vez: o cliente paga sempre ao empregado ou no caixa.

Como o pedido anda de mesa em mesa

O pedido nasce já com a mesa colada, porque a mesa vem do QR Code e não de algo que o cliente escreve. Por isso não há confusão entre mesas.

1. O cliente lê o QR da Mesa 3. O sistema descobre a mesa pelo código secreto do link.
2. Escolhe os pratos, junta notas e toca em **Enviar pedido**.
3. O sistema grava o pedido com: mesa, itens, quantidades, notas, total e hora.
4. O painel do administrador recebe o pedido na hora, toca o alerta e mostra-o no cartão da Mesa 3.
5. O administrador muda o estado: **Recebido**, depois **Em preparação**, depois **Entregue**. O cliente vê essa mudança no telemóvel.
6. Se a Mesa 3 pedir de novo, o novo pedido junta-se aos anteriores da mesma sessão da mesa.
7. Quando o cliente paga, o administrador toca em **Fechar mesa**. O histórico fica guardado e a mesa volta a ficar livre.

Regras que o sistema tem de cumprir:

- Um pedido da Mesa 3 nunca aparece noutra mesa.
- O cliente só vê os pedidos da sua própria mesa, e só os da sessão atual.
- Se o mesmo código de QR for usado por alguém fora do restaurante, o pedido continua a ser da Mesa 3; por isso o administrador pode cancelar um pedido suspeito.
- Depois de **Fechar mesa**, o QR continua a funcionar, mas abre uma sessão nova e vazia.

Modelo de dados

Seis tabelas chegam. A "sessão" é o que separa o cliente de agora do cliente de há duas horas na mesma mesa.

Tabela	Campos principais
`restaurant_tables`	id, número, nome, `qr_token` (único e secreto), ativa
`table_sessions`	id, mesa, aberta_em, fechada_em, estado (aberta ou fechada)
`categories`	id, nome, ordem
`menu_items`	id, categoria, nome, descrição, preço, foto, disponível, ordem
`orders`	id, sessão, mesa, estado (recebido, em preparação, entregue, cancelado), nota geral, total, criado_em
`order_items`	id, pedido, prato, nome e preço copiados no momento, quantidade, nota

O nome e o preço do prato são copiados para `order_items` no momento do pedido. Assim, se o preço mudar amanhã, o histórico de hoje continua certo.

Segurança: o cliente nunca escreve direto na base. O pedido passa por uma função segura da própria base (chamada com supabase.rpc) que valida o `qr_token`, descobre a mesa e só então grava. O painel do administrador só funciona com login, e a base usa regras de acesso (RLS) para bloquear tudo o resto.

Plano de construção

O Claude Code deve trabalhar por fases e testar cada uma antes de passar à seguinte. Assim os erros aparecem cedo.

#	Fase	Fica pronto	Como confirmar
1	Base	Projeto, Supabase, tabelas, regras de acesso, cardápio de exemplo, 10 mesas	As tabelas existem e têm dados
2	Cliente	Página `/m/:token` com cardápio e carrinho	Abrir o link no telemóvel e montar um carrinho

#	Fase	Fica pronto	Como confirmar
3	Pedido	Envio do pedido para a mesa certa, ecrã de confirmação, estado do pedido	Pedido da Mesa 1 grava com a mesa 1
4	Painel	Login, cartões das mesas, detalhe, estados, tempo real com som	Pedido enviado num telemóvel aparece no painel sem atualizar
5	Gestão	Cardápio editável, adicionar e remover mesas, página de QR para imprimir	Criar a Mesa 11 e imprimir o QR dela
6	Entrega	Testes com 3 mesas ao mesmo tempo, correção de erros, deploy na Vercel	Link público a funcionar nos telemóveis do restaurante

Depois da versão 1, os próximos passos mais úteis são contas separadas para cozinha e empregados, impressão automática do talão e relatório de vendas do dia. Pagamento online não entra no sistema.

Prompt para colar no Claude Code

Copia tudo o que está dentro da caixa abaixo e cola numa pasta nova, com o Claude Code aberto. Se já tiveres o cardápio real (pratos e preços), cola-o a seguir ao prompt.

```
És um engenheiro full-stack sénior. Constrói o sistema descrito abaixo, com a interface toda em português de Moçambique. Antes de escrever código, lê o prompt inteiro e mostra um plano curto. Depois constrói por fases e testa cada fase antes de passar à seguinte.

# Objetivo
Sistema de pedidos por QR Code para o restaurante "Bom Paladar". Cada mesa tem um QR Code próprio. O cliente lê o QR com o telemóvel, vê o cardápio, escolhe pratos, escreve notas e envia o pedido. O pedido chega em tempo real a um painel de administrador, já identificado pela mesa. O cliente não instala app e não cria conta. O sistema não processa pagamentos de nenhum tipo: o cliente paga no fim, ao empregado ou no caixa.

# Stack
- Vite + React + TypeScript + React Router + Tailwind CSS (SPA, sem Next.js e sem backend próprio)
- Supabase: Postgres, Auth (email e senha), Realtime, Storage
- Deploy na Vercel como site estático, com um vercel.json que redireciona todas as rotas para /index.html (para o React Router funcionar)
- Variáveis de ambiente: VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY,
```

```
VITE_SITE_URL. A service role key nunca entra no projeto. Nunca escrevas
chaves no código nem no Git. Cria um .env.example. Se faltar uma chave, pára
e pede-ma.
- Moeda: Meticais (MT). Fuso horário: Africa/Maputo. Horas em formato 24h
(HH:mm).

# Base de dados
Escreve migrações SQL em supabase/migrations (e aplica-as se tiveres acesso
ao Supabase). Tabelas:
- restaurant_tables: id, numero (int, único), nome, qr_token (texto
aleatório, único, 10+ caracteres), ativa (bool), criado_em
- table_sessions: id, table_id, aberta_em, fechada_em (nulo enquanto aberta).
Só pode existir uma sessão aberta por mesa.
- categories: id, nome, ordem
- menu_items: id, category_id, nome, descricao, preco (numeric), foto_url,
disponivel (bool), ordem
- orders: id, session_id, table_id, estado (recebido | em_preparacao |
entregue | cancelado), nota_geral, total, criado_em, atualizado_em
- order_items: id, order_id, menu_item_id, nome e preco_unitario copiados no
momento do pedido, quantidade, nota

Segurança (RLS ativa em todas as tabelas):
- O visitante anónimo só pode LER categories e menu_items (cardápio público).
Não lê nem escreve mais nada diretamente.
- Todas as operações do cliente passam por funções SQL SECURITY DEFINER
chamadas com supabase.rpc (get_table_info, create_order, get_table_orders),
que validam o qr_token. Não há backend próprio. Dá EXECUTE a essas funções só
ao papel anon e fixa o search_path de cada uma.
- O administrador autenticado tem acesso total.
- Desativa o registo público de utilizadores. O admin é criado à mão no
Supabase Auth; explica isto no README.

Cria também get_table_info(token) e get_table_orders(token), para o cliente
ler o nome da mesa e os pedidos da sessão aberta. Cria a função SQL
create_order(token, items jsonb, nota text) que, numa só transação: encontra
a mesa pelo token (e rejeita se for inválido ou a mesa estiver inativa), abre
uma sessão se não houver nenhuma aberta, valida que cada prato existe e está
disponível, lê os preços da base (nunca confies no preço vindo do cliente),
calcula o total e grava orders e order_items.

Seed: 10 mesas (números 1 a 10, cada uma com qr_token aleatório) e um
cardápio de exemplo com 4 categorias (Entradas, Pratos principais, Bebidas,
Sobremesas) e cerca de 15 pratos com preços em MT. Se eu colar o cardápio
real, usa esse em vez do exemplo.

# Parte do cliente (pública, mobile-first)
```

```
- Rota /m/:token (React Router). Token inválido mostra uma página amigável "QR Code inválido, chame um empregado".
- Topo fixo com o nome do restaurante e "Mesa N".
- Cardápio por categorias, com navegação rápida entre elas. Cada prato mostra foto, nome, descrição e preço. Pratos esgotados aparecem a cinzento e não se podem pedir.
- Ao tocar num prato abre uma folha (bottom sheet) com quantidade e campo de nota (ex.: sem cebola, sem gelo).
- Carrinho fixo em baixo com o número de itens e o total. Ecrã de resumo onde se pode alterar quantidades, apagar itens e escrever uma nota geral. Botão grande "Enviar pedido".
- Depois de enviar: ecrã de confirmação, carrinho esvaziado, e a opção de pedir mais.
- Separador "Os meus pedidos": mostra os pedidos da sessão atual da mesa, com hora, itens, notas, total e estado. Atualiza sozinho a cada 10 segundos (polling simples com get_table_orders, sem expor Realtime ao anónimo).
- Proteção contra spam: no máximo 5 pedidos por mesa por minuto, verificado dentro de create_order; evita duplo clique no botão de enviar.
- Botões e áreas de toque grandes, carregamento e estados vazios bem tratados.

# Painel do administrador (protegido por login)
- /admin/login com email e senha (Supabase Auth). Todas as rotas /admin exigem sessão, verificada num componente ProtectedRoute; o acesso real aos dados é garantido pelo RLS.
- /admin: grelha de cartões, um por mesa ativa (Mesa 1 a Mesa 10 e as que forem adicionadas). Cada cartão mostra se a mesa está livre ou ocupada, o número de pedidos por entregar, o total em aberto e a hora do último pedido. Mesas com pedidos novos ficam destacadas.
- Tempo real: o painel subscreve a tabela orders com Supabase Realtime. Quando chega um pedido novo, toca um som e mostra um aviso no topo. Como os navegadores bloqueiam som sem interação, inclui um botão "Ativar som".
- /admin/mesas/:id: detalhe da mesa com todos os pedidos da sessão aberta. Cada pedido mostra a hora, os itens com quantidade e nota (destacada), a nota geral, o total e o estado. Botões para avançar o estado (Recebido, Em preparação, Entregue) e para cancelar. Botão "Fechar mesa" que fecha a sessão e deixa a mesa livre, pedindo confirmação.
- /admin/historico: todos os pedidos, mais recentes primeiro, com filtro por mesa e por data, mostrando hora, mesa, itens, notas, estado e total.
- /admin/mesas: adicionar mesa (próximo número automático), renomear, desativar e apagar (só se não tiver pedidos). Mostra o QR Code de cada mesa.
- /admin/qrcodes: página pensada para imprimir, com um cartão por mesa (nome da mesa grande, QR Code, texto "Aponte a câmara para ver o cardápio"). O QR codifica VITE_SITE_URL + /m/ + qr_token. Usa a biblioteca qrcode e CSS de impressão (@media print). Botão para regenerar o token de uma mesa.
- /admin/cardapio: CRUD de categorias e pratos, upload de foto para o
```

```
Supabase Storage, marcar prato como esgotado, reordenar.

# Regras de negócio
- A mesa de um pedido vem sempre do qr_token, nunca de um campo que o cliente escreva.
- Um pedido nunca aparece numa mesa diferente da sua. O cliente só vê pedidos da sua mesa e da sessão aberta.
- Depois de "Fechar mesa", o mesmo QR continua a funcionar mas abre uma sessão nova e vazia.
- O total em aberto de uma mesa exclui pedidos cancelados.
- Preço e nome dos pratos são copiados para order_items no momento do pedido, para o histórico não mudar quando o cardápio mudar.
- Todas as horas são guardadas em UTC (timestamptz) e mostradas em Africa/Maputo.

# Qualidade
- Código limpo, com tipos TypeScript, validação das entradas dos formulários (por exemplo com zod) e mensagens de erro em português.
- Interface limpa, rápida e agradável, com cores quentes de restaurante. O painel deve ser legível num tablet e num portátil.
- Escreve testes para a lógica crítica: create_order com token inválido, prato esgotado, preço adulterado pelo cliente e isolamento entre mesas.
- README em português com: como criar o projeto Supabase, como correr as migrações, como criar o utilizador admin, variáveis de ambiente, como correr localmente e como fazer deploy na Vercel.

# Método de trabalho
1. Mostra o plano.
2. Fase 1: projeto, Supabase, migrações, RLS, seed.
3. Fase 2: página do cliente com cardápio e carrinho.
4. Fase 3: envio do pedido, confirmação e "Os meus pedidos".
5. Fase 4: login do admin, grelha de mesas, detalhe da mesa, estados e tempo real com som.
6. Fase 5: gestão de cardápio, gestão de mesas e página de QR Codes.
7. Fase 6: testes, correções, build de produção (npm run build sem erros), README e deploy.
No fim de cada fase, corre o build, testa o que construíste e diz-me em duas linhas o que ficou pronto e como eu posso confirmar. Faz um commit por fase.

# Fora do âmbito desta versão
Pagamento online está PROIBIDO: não o construas, não integres M-Pesa, e-Mola, cartões nem gateways, e não deixes botões ou campos de pagamento na interface. O cliente paga sempre ao empregado ou no caixa. Também ficam de fora: contas para empregados ou cozinha, impressão de talão e relatórios.
```

Depois de o Claude Code terminar, a primeira coisa a testar é abrir o link de duas mesas em dois telemóveis e confirmar que cada pedido cai só na mesa certa.